

Étude Comparative des Bases de Données Vectorielles

Choix d'une solution d'indexation et de recherche de similarité pour un chatbot juridique RAG destiné aux PME

Projet : MVP Chatbot d'Assistance Juridique (Droit du travail & OHADA) — OS3 : Modèle

Auteur : Robin Yonli

1. Qu'est-ce qu'une base de données vectorielle ?

Une base de données vectorielle est un système de stockage conçu pour manipuler des **embeddings** : des listes de nombres (vecteurs) qui représentent le sens d'un texte, d'une image ou de tout autre contenu. Contrairement à une base de données classique (SQL), qui recherche des correspondances exactes (un mot, un identifiant), une base vectorielle recherche des contenus qui se **ressemblent par le sens**, même s'ils n'utilisent pas les mêmes mots.

Exemple concret pour notre projet : si un utilisateur pose la question « Mon patron peut-il me licencier sans préavis ? », le chatbot doit retrouver les articles du Code du travail qui parlent de « rupture du contrat », de « préavis de licenciement » ou de « congédiement », même si le mot exact « patron » n'apparaît jamais dans ces textes. C'est exactement le rôle d'une base vectorielle dans une architecture RAG (Retrieval-Augmented Generation) : elle constitue la mémoire dans laquelle le chatbot va chercher les passages juridiques les plus pertinents avant de répondre.

2. Fonctionnement : indexation et recherche de similarité

2.1 L'indexation

Quand un document juridique est ajouté à la base, il est d'abord découpé en petits morceaux (« chunks »), puis chaque morceau est transformé en vecteur par un modèle d'embedding. L'**indexation** consiste à organiser tous ces vecteurs dans une structure qui permettra de les retrouver rapidement plus tard, sans avoir à comparer un par un les millions de vecteurs possibles. C'est l'équivalent d'un index de livre : au lieu de relire tout l'ouvrage, on consulte une structure qui pointe directement vers les bonnes pages.

Les bases vectorielles modernes utilisent le plus souvent des index de type **HNSW (Hierarchical Navigable Small World)**, qui construisent une sorte de carte en couches permettant de sauter rapidement vers les vecteurs voisins, plutôt qu'un index de type IVF (qui regroupe les vecteurs en « clusters » à explorer).

2.2 La recherche de similarité

Lorsqu'un utilisateur pose une question, celle-ci est elle aussi transformée en vecteur. La base vectorielle calcule alors la **distance** entre ce vecteur-question et tous les vecteurs déjà indexés, puis renvoie les plus proches. La mesure la plus utilisée est la **similarité cosinus** : elle mesure l'angle entre deux vecteurs plutôt que leur taille, ce qui permet de comparer des textes de longueurs différentes de façon fiable. Une similarité cosinus de 1 signifie que les deux textes ont exactement le même sens vectoriel ; une similarité proche de 0 signifie qu'ils n'ont presque aucun rapport.

D'autres distances existent (distance euclidienne, produit scalaire), mais la similarité cosinus reste la référence pour les cas d'usage textuels comme le nôtre, car elle est peu sensible à la longueur des passages juridiques comparés.

3. Recensement des solutions existantes

Cinq solutions de référence ont été étudiées : FAISS, ChromaDB, Pinecone, Milvus et Qdrant. Le tableau ci-dessous synthétise leurs avantages, inconvénients, coûts et complexité d'installation, dans une optique d'adoption par une petite ou moyenne entreprise (PME).

Solution	Avantages	Inconvénients	Coût	Complexité
FAISS (Meta) ✓ RETENU	Bibliothèque de référence en recherche de similarité ; performance brute inégalée ; gratuit et open-source ; support GPU natif ; 100% interne (aucune donnée transmise à un tiers).	Pas de gestion native des métadonnées ni de persistance : nécessite une fine couche applicative développée par l'équipe (faite dans ce projet).	Gratuit	Maîtrisée (couche développée en interne)
ChromaDB	Très simple à installer (une ligne de code) ; pensé pour le RAG et LangChain ; gestion native des métadonnées et filtres.	Moins mature pour de très gros volumes ; écosystème cloud encore jeune.	Gratuit (self-hosted)	Faible
Pinecone	Service entièrement managé (aucun serveur à gérer) ; très scalable ; bonne fiabilité en production.	Solution propriétaire et payante au-delà d'un certain volume ; dépendance à un service cloud externe (donnée hébergée hors PME).	Payant (offre gratuite limitée)	Faible
Milvus	Conçu pour des volumes massifs (milliards de vecteurs) ; très performant ; communauté active.	Architecture lourde (plusieurs composants : etcd, MinIO...) ; surdimensionné pour un MVP.	Gratuit (self-hosted)	Élevée
Qdrant	Bon compromis simplicité/performance ; écrit en Rust (rapide) ; API claire ; bonne gestion des filtres sur métadonnées ; option cloud gratuite limitée.	Communauté plus petite que Pinecone ou Milvus ; moins d'intégrations tierces.	Gratuit (self-hosted) ou cloud freemium	Faible à moyenne

4. Conclusion : choix recommandé pour le projet

Pour un MVP destiné à des PME manipulant des données juridiques sensibles, trois critères dominant : un **contrôle total sur l'hébergement des données** (aucune donnée juridique de PME ne doit transiter par un service cloud tiers), un **coût nul** (pas de licence ni d'abonnement récurrent), et une **performance maximale** de la recherche de similarité, le cœur du moteur RAG.

Pinecone est écarté d'emblée : c'est un service propriétaire payant qui héberge les vecteurs hors de l'infrastructure de la PME, ce qui pose un problème direct de confidentialité pour des données juridiques. **Milvus** est écarté car son architecture (etcd, MinIO, plusieurs services) est trop lourde à déployer et à maintenir pour une petite équipe. **ChromaDB** et **Qdrant** restent de bonnes options « tout-en-un », mais elles ajoutent une couche serveur supplémentaire et restent, à volume égal, moins rapides que des index optimisés bas niveau.

Recommandation : FAISS (Facebook AI Similarity Search, Meta). FAISS est la bibliothèque de référence en recherche de similarité vectorielle : elle a été conçue et optimisée par les équipes de recherche de Meta spécifiquement pour offrir la meilleure performance brute sur des index de type HNSW et IVF, avec un support GPU natif si besoin. Étant une bibliothèque embarquée directement dans le code Python du projet (et non un service externe), elle s'intègre parfaitement dans une infrastructure 100% interne : aucune dépendance réseau, aucune donnée juridique transmise à un tiers, et aucun coût de licence. C'est ce niveau de contrôle et de performance qui en fait le choix retenu pour le moteur de recherche vectorielle du chatbot.

Le seul point de vigilance est que FAISS ne gère pas nativement la persistance ni les métadonnées : l'équipe a donc développé une fine couche applicative autour de FAISS (sauvegarde de l'index sur disque, stockage des métadonnées des articles juridiques dans une structure associée) pour obtenir exactement les fonctionnalités nécessaires, sans le poids d'un serveur de base de données complet.

En résumé : **FAISS retenu pour sa performance, sa gratuité et le contrôle total qu'il offre sur l'hébergement des données juridiques, avec une légère couche applicative développée par l'équipe pour compenser l'absence de persistance et de métadonnées natives.**